

Sets

- A **Set** is a collection of **unique items** — no duplicates allowed!
- It is **unordered** (no fixed position) and **mutable** (can add or remove items).
- A set is a collection which is unordered and unindexed.

Syntax:

```
set1 = {"abc", 34, True, 40, "male"}
```

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {1, 5, 7, 9, 3}
```

```
set3 = {True, False, False}
```

Set in Python

$S = \{ 20, 'Jessa', 35.75 \}$

- ✓ **Unordered:** Set doesn't maintain the order of the data insertion.
- ✓ **Heterogeneous:** Set can contains data of all types
- ✓ **Unique:** Set doesn't allows duplicates items

Example :

```
fruits = {"apple", "banana", "apple", "orange"}  
  
print(fruits)
```

Output:

```
{'apple', 'banana', 'orange'}
```

Note : Even if you add “apple” twice, it appears only once — because sets don’t allow duplicates!

Property	Description
Unordered	The elements have no defined order. You can't access elements using an index.
Unique elements	Duplicates are automatically removed.
Mutable (changeable)	You can add or remove elements after creation.
Heterogeneous elements allowed	It can contain different data types (like integers, strings, etc.).
Immutable elements not allowed	Items inside must be immutable (like numbers, strings, tuples). Lists and dictionaries are not allowed.

Note : Set items are unchangeable via index , but you can remove items and add new items.

```
fruits = {"apple", "banana", "mango"}
```

You CANNOT do this:

```
fruits[0] = "grape"  # Error! sets have no index, and items can't be changed
```

But you **CAN** do this :

```
fruits.remove("banana")  # remove an item
```

```
fruits.add("grape")      # add a new one
```

```
print(fruits)
```

Output:

```
{'apple', 'grape', 'mango'}
```

What is Hashing?

Hashing = Making a shortcut number (hash value) for every item.

Python uses a special formula (called the **hash function**) to convert your item into a unique number.

Example:

```
print(hash(10))      # might show: 10
```

```
print(hash("apple")) # might show: 2345678901234
```

So, `10` and `"apple"` each get their own **hash number**.

How a Set Uses Hashing :

Let's say you write:

```
s = {10, 20, 30}
```

- Python needs to **store** these numbers somewhere in memory.
- But instead of storing them in a simple list or array (like `[10, 20, 30]`),
- it uses a **hash table** — a special structure that makes searching super **fast** .

Python Creates a Hash Table

- Think of a **hash table** like a row of boxes (called *buckets*).
- Each bucket has an **index number** (like 0, 1, 2, 3...).

Example (just 8 buckets for simplicity):

Bucket index	Value stored
0	—
1	—
2	—
3	—
4	—
5	—
6	—
7	—

Python Calculates a “Hash Value” :

When you do:

s.add(10)

Python calls:

hash(10) #Suppose it gives a big number (like **123456789**).

Now Python needs to decide *which bucket* to store it in.

Formula : $\text{index} = \text{hash}(\text{value}) \% \text{number_of_buckets}$

If the table has 8 buckets:

$\text{index} = 123456789 \% 8 = 5$

So, **10** goes into **bucket 5**.

Now the table looks like:

Bucket index	Value stored
0	—
1	—
2	—
3	—
4	—
5	10
6	—
7	—

Searching an Element :

When you check:

if 10 in s:

Python doesn't loop through all values.
Instead, it directly calculates:

hash(10) → 123456789

index = 123456789 % 8 = 5

and looks straight into **bucket 5**.

If **10** is there → found !

That's how **searching in a set** is so fast — usually constant time (**O(1)**).

Why Only Immutable (Unchangeable) Items Allowed?

If you put something **changeable** (like a list) inside a set, its hash value could change when the list changes.

Example:

`hash([1, 2, 3])` → say 105 # later if we change it

`my_list.append(4)` # list is now [1, 2, 3, 4]

`hash([1, 2, 3, 4])` → 207

But Python has **already stored it at locker 105!**

Now it doesn't know where to find it or where to put new ones

The whole hash table gets confused!

That's why Python says: “Sorry, you can't put mutable items inside a set.”

Creating a Set : Syntax & Examples

Using curly braces
{ }

```
my_set = {1, 2, 3, 4}
```

Using the set() constructor

```
numbers = set([1, 2, 2, 3])
```

Use set() for an empty set.

```
empty_set = set()
```

⚠ Note: An empty {} creates a **dictionary**, not a set.

Accessing Set Elements

Since sets are unordered, you cannot access elements by index.

❓ Wrong:

Right:



```
my_set = {1, 2, 3}
print(my_set[0]) # Error!
```

❓



```
for item in my_set:
    print(item)
```

Adding Elements

1. Add Elements



```
s = {1, 2, 3}  
s.add(4)
```

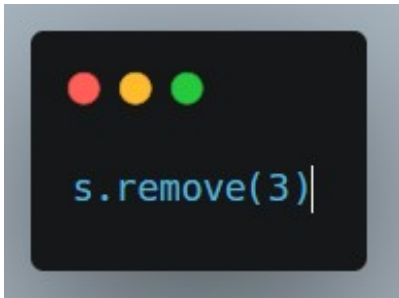
2. Add Multiple Elements



```
s.update([5, 6, 7])
```

Removing Elements

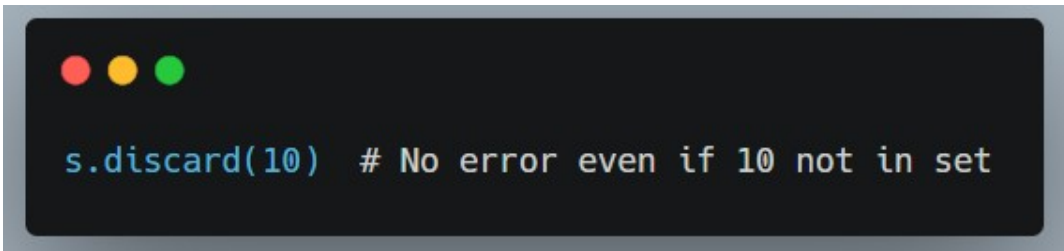
1. Remove Elements



```
s.remove(3)|
```

⚠ `remove()` throws an error if the element doesn't exist.

2. Discard Elements

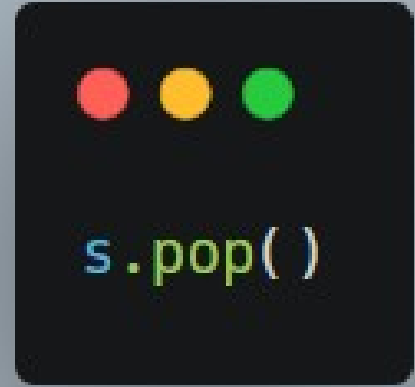


```
s.discard(10) # No error even if 10 not in set
```

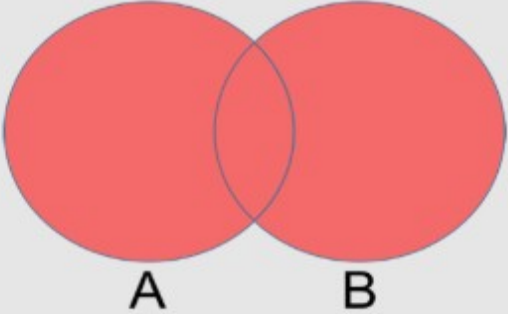
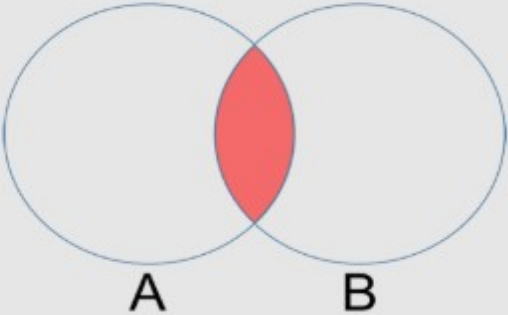

Removing Elements

3. Remove and Return a Random Element

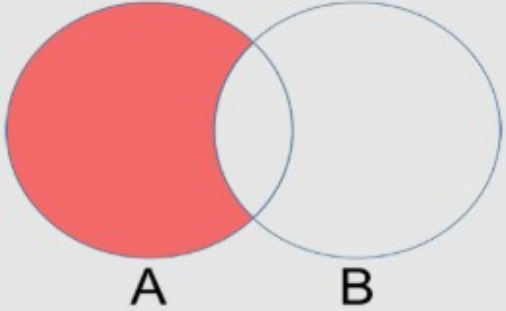
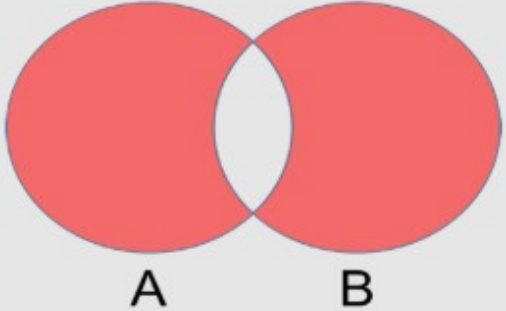
4. Clear All Elements



Set Operations (Mathematical)

Set Operation	Venn Diagram	Interpretation
Union	 A Venn diagram showing two overlapping circles, labeled A and B. Both circles are filled with a solid red color, representing the union of the two sets.	$A \cup B$, is the set of all values that are a member of A, or B, or both.
Intersection	 A Venn diagram showing two overlapping circles, labeled A and B. Only the overlapping region between the two circles is shaded red, representing the intersection of the two sets.	$A \cap B$, is the set of all values that are members of both A and B.

Set Operations (Mathematical)

Set Operation	Venn Diagram	Interpretation
Difference	 A Venn diagram showing two overlapping circles, A and B. Circle A is shaded red, and circle B is white. The intersection of A and B is also white.	$A \setminus B$, is the set of all values of A that are not members of B
Symmetric Difference	 A Venn diagram showing two overlapping circles, A and B. Both circles are shaded red, but the intersection of A and B is white.	$A \triangle B$, is the set of all values which are in one of the sets, but not both.

Set Operations (Mathematical)

1. Union ($A \cup B$):

All unique elements from both sets.



```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B)           # {1, 2, 3, 4, 5, 6}
print(A.union(B))      # {1, 2, 3, 4, 5, 6}
```

Set Operations (Mathematical)

2. Intersection ($A \cap B$):

Elements common to both sets.



```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A & B)           # {3, 4}
print(A.intersection(B)) # {3, 4}
```

Set Operations (Mathematical)

3. Difference ($A - B$):

Elements in A but not in B.



```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A - B)           # {1, 2}
print(A.difference(B)) # {1, 2}
```

Set Operations (Mathematical)

4. Symmetric Difference ($A \triangle B$)

Elements in A and B but not in both.

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A ^ B)                # {1, 2, 5, 6}
print(A.symmetric_difference(B)) # {1, 2, 5, 6}
```

Set Comparison Methods

Method	Description	Example
<code>issubset()</code>	Checks if all elements of one set are in another	<code>{1, 2}.issubset({1, 2, 3}) → True</code>
<code>issuperset()</code>	Checks if one set contains all elements of another	<code>{1, 2, 3}.issuperset({1, 2}) → True</code>
<code>isdisjoint()</code>	Checks if two sets have no elements in common	<code>{1, 2}.isdisjoint({3, 4}) → True</code>

Useful Built-in Functions with Sets

Function	Description	Example
<code>len(s)</code>	Returns number of elements	<code>len({1, 2, 3}) → 3</code>
<code>max(s)</code>	Largest element	<code>max({1, 5, 3}) → 5</code>
<code>min(s)</code>	Smallest element	<code>min({1, 5, 3}) → 1</code>
<code>sum(s)</code>	Sum of all elements (if numeric)	<code>sum({1, 2, 3}) → 6</code>
<code>sorted(s)</code>	Returns a sorted list of elements	<code>sorted({3, 1, 2}) → [1, 2, 3]</code>
<code>in</code>	Check membership	<code>2 in {1, 2, 3} → True</code>

Frozen Sets (Immutable Sets)

- If you want a **set that cannot be changed**, use a **frozenset**.
- You can perform union, intersection, etc., but **cannot** add or remove elements. 



```
fs = frozenset((1,2,3,4,5))
print(type(fs))    # <class 'frozenset'>
print(fs)          # frozenset({1, 2, 3, 4, 5})
```

Set practice Questions :



1. Create an **empty set** correctly. (Hint: **{}** or **set()**?)
2. Add a new element **6** to the set **{1, 2, 3, 4, 5}**.
3. Remove the element **3** from **{1, 2, 3, 4, 5}**.
4. Try removing an element that doesn't exist using **.remove()** and **.discard()**.
→ What difference do you notice?
5. Use **len()** to find the number of elements in a set.
6. Check if **4** is present in **{2, 4, 6, 8}** using the **in** keyword.

Create two sets:

A = {1, 2, 3, 4}

B = {3, 4, 5, 6}

9. Perform the following:

- Union
- Intersection
- Difference (A – B)
- Symmetric Difference

10. Use built-in operators to do the same (`|`, `&`, `-`, `^`).

11. Check if set **{1, 2}** is a **subset** of **{1, 2, 3, 4}**.

12. Check if **{1, 2, 3, 4}** is a **superset** of **{2, 3}**.

13. Use **.clear()** to empty a set. What happens if you print it afterward?

15. Write a program that reads two lists of student roll numbers and prints:
- Students who are in **both** lists
 - Students who are in **only one** list
16. Find all **unique characters** in a string entered by the user.

Input: "programming"

Output: {'r', 'a', 'm', 'p', 'i', 'o', 'g', 'n'}

17. Given two sets of subjects:

science = {"Physics", "Chemistry", "Biology"}

commerce = {"Accountancy", "Economics", "Math"}

Check if they have any common subject.

18. Remove all **duplicate numbers** from a list using a set.

nums = [1, 2, 2, 3, 3, 3]

Output: {1, 2, 3}

19. Convert the string **"python"** into a set and print the result.
→ What happens to repeated letters?